

closure_evidence_20260925

BOB-187 closure evidence — 2026-09-25

Revision: 1 Last modified: 2026-09-25T11:31:05Z

The defect (verbatim, from the tracker)

scripts/ownership_precondition.sh consumes the SAME unreviewed .env-driven scope as ownership_repair.sh and creates probe files inside it, but it is NOT fenced. T028 remediation added ownership_path_fence() to scripts/lib/ownership.sh and wired it into ownership_repair.sh, closing the path-escape class there; the precondition was outside that agent's file scope and still accepts whatever config/owned_paths.yaml expands to. Blast radius is lower than a recursive chown (it writes probe files rather than mutating ownership of an existing tree) but the INPUT is identical and equally unreviewed, so the same QBITTORRENT_DATA_DIR value that would have driven a filesystem-wide chown will drive probe-file creation at an arbitrary location.

Acceptance: ownership_precondition.sh calls the SAME ownership_path_fence() predicate from scripts/lib/ownership.sh (never a second dialect, §11.4.251), refuses fail-closed on a rejected entry, and ships a paired §1.1 mutation proving the refusal fires PLUS a negative control proving the six live shipped entries still ACCEPT so the fix is not a §11.4.201(1) false-positive refusal.

Investigation finding — the fix was ALREADY LANDED

`git log --oneline -- scripts/ownership_precondition.sh` shows the fence was wired in by commit `af48019` (“feat(BOB-183,BOB-187): a freshness gate for the served bundle, and a fence for the precondition’s scope”, 2026-08-25), BEFORE this session began. `scripts/ownership_precondition.sh main()` (lines 1082–1138) already calls `ownership_path_fence()` — the exact same function `scripts/ownership_repair.sh` calls — before any probe file is created, and `tests/unit/test_ownership_precondition.sh Case 6 (6a–6d, lines 416–592)` already covers:

- 6a: an absolute entry naming a system tree (`/etc`) → exit 2, refused
- 6b: the filesystem root (`/`) → exit 2, refused, reason named
- 6c: a repo-relative `..` escape (the exact class the item text describes) → exit 2, refused, **and a real state-delta assertion** — the escaped directory’s mtime is proven UNCHANGED, i.e. `probe_location()` never ran there
- 6d: the mandatory §11.4.201(1) negative control — the six entries actually shipped in `config/owned_paths.yaml` all ACCEPT through the fence, checked across three real `QBITTORRENT_DATA_DIR` values (unset/shipped-default, a deep `/run/media/...` shape, and this host’s real `.env` value)

Running the suite unmodified, before any change in this session:

```
$ timeout 300 bash tests/unit/test_ownership_precondition.sh 2>&1
| tail -8
PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=<unset, shipped default /mnt/DATA>
PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=/run/media/operator/DISK4TB/Downloads
PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=/home/milosvasic/Share/Misc
RESULT: 29 passed, 0 failed, 0 skipped
```

The gap was exactly the item text’s second acceptance clause: a “paired §1.1 mutation proving the refusal fires” was performed manually during `T028/af48019`’s development (commit message: “M1 (fence call disabled) -> 6 fence assertions fail again. M2 (fence refuses everything) -> 14 failed”) but was **never landed as a permanent, re-runnable regression test** — only the negative control (6d) and the RED-reproduction-by-construction (6c) were committed. Without a standing mutation case, a future edit that silently detached the fence call from `main()` (e.g. an accidental revert of the `if ! f_reason="$`

(ownership_path_fence ...) " line) would leave Case 6a/6b/6c passing for the WRONG reason (any other bug that happens to also cannot_run the scope) and 6d passing trivially, with nothing in the suite to catch the detachment itself. This is the coverage escape the item text names (§11.4.238): the fix existed, its confirming mutation did not, out-of-band.

The work performed this session

Added **Case 6e** to tests/unit/test_ownership_precondition.sh (lines 593–692): a permanent, hermetic, paired §1.1 mutation test that:

1. Builds a **sandboxed copy** of scripts/ownership_precondition.sh + scripts/lib/ownership.sh under \$FIX (mirroring the sb_new() pattern already established in tests/unit/test_ownership_repair.sh), sha256- verified byte-identical to the tracked source before mutation.
2. **(i) Pre-flight GREEN:** runs the SAME ..-escape scenario as Case 6c against the UNMUTATED sandbox copy, confirming it reproduces 6c's exit-2 refusal and unchanged mtime — proving the sandbox is a faithful stand-in for the real artifact before any mutation is applied.
3. **(ii)** Verifies the sed target (f_reason="\$(ownership_path_fence) is the exactly-one call site in main(), refusing an ambiguous mutation if a future refactor changes the call shape.
4. **(iii) The mutation:** sed-replaces ownership_path_fence with true at that one call site (never touching the tracked file — only the sandboxed copy), then re-runs the identical ..-escape scenario against a FRESH escape-target directory. Asserts the escape now SUCCEEDS (exit 0, OWNERSHIP-PRECONDITION: OK, and — the load-bearing check — the escaped directory's mtime CHANGES, proving probe_location() really created and removed a file outside the sandbox's own project root).
5. No restore step is needed: only the sandboxed copy is ever mutated; the tracked scripts/ownership_precondition.sh is never written to. \$FIX (and everything under it, including the sandbox) is reaped by the suite's own trap ... EXIT.

RED (mutation proves the test is load-bearing)

Scratch verification (before landing the test), reproducing the escape on a scoped, safe fixture directory — never against a real system path — using a sandboxed copy of the artifact with the fence call neutered:

```
=== red (mutate=yes) ===
RC=0
mtime CHANGED -> escape SUCCEEDED (probe file was created)
OWNERSHIP-PRECONDITION: OK
```

Compare against the same scenario on the unmutated sandbox copy:

```
=== green (mutate=no) ===
RC=2
mtime UNCHANGED -> escape refused
OWNERSHIP-PRECONDITION: CANNOT-RUN
  - the declared scope names 1 path(s) this check must never
probe: ...
  - REFUSED ...escape_target_green - '...' was declared as a repo-
relative
    path but resolves OUTSIDE the project root '...' - a relative
entry may
    not climb out with '..'
  - nothing was probed, and no probe file was created anywhere
This check asserted NOTHING. It is not a pass - fix the cause
above and re-run.
```

Suite-level mutation proof (the real artifact, in-place, then restored)

To close the loop with a literal in-place mutation of the tracked file (not only the sandboxed copy the permanent test uses), the fence call was temporarily neutered on disk and the FULL suite re-run:

```
$ sha256sum scripts/ownership_precondition.sh
f7e48c2394d5b2d7790497b8a47ca909bc2272774c082a4b80d3abb2ce6a72d7
scripts/ownership_precondition.sh

$ sed -i 's/f_reason="\$(ownership_path_fence /f_reason="\$(true /'
scripts/ownership_precondition.sh

$ timeout 300 bash tests/unit/test_ownership_precondition.sh 2>&1
| tail -12
  FAIL: mutation harness pre-flight: unmutated sandbox copy did
NOT refuse the
```

```
    '..' escape (rc=0) – the sandbox does not faithfully
reproduce Case 6c,
    so a RED result below would not be trustworthy
    FAIL: mutation harness: expected exactly 1 fence call site in
main(), found
    0 – the sed target is no longer unique, mutation would be
ambiguous
RESULT: 23 passed, 8 failed, 0 skipped
```

(8 failures: Cases 6a/6b/6c themselves plus their sub-assertions, and the new Case 6e's own pre-flight/uniqueness guards — since the mutation harness copies the ALREADY-mutated real script into its own sandbox, it correctly detects the double-mutated, already-broken state as well.)

Restored and re-verified:

```
$ cp -p <backup> scripts/ownership_precondition.sh
$ sha256sum scripts/ownership_precondition.sh
f7e48c2394d5b2d7790497b8a47ca909bc2272774c082a4b80d3abb2ce6a72d7
scripts/ownership_precondition.sh  # byte-identical to before

$ git diff --stat scripts/ownership_precondition.sh
(empty – no diff; file returned to its committed state)

$ timeout 300 bash tests/unit/test_ownership_precondition.sh 2>&1
| tail -6
    PASS: mutation harness pre-flight: unmutated sandbox copy
refuses the '..'
        escape exactly like the real artifact (exit 2, mtime
unchanged) – the
        sandbox is a faithful stand-in
    PASS: mutation harness: the fence call site was neutered
        (ownership_path_fence -> true) in the sandbox copy
    PASS: mutation: fence call neutered -> the SAME '..' escape now
creates
        (and removes) a probe file OUTSIDE the sandbox project
root (mtime
        changed, rc=0) – Case 6c's PASS is proven load-bearing,
not decoration
RESULT: 32 passed, 0 failed, 0 skipped
```

Negative control (already present, re-confirmed unmodified — Case 6d)

```
PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=<unset, shipped default /mnt/DATA>
PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=/run/media/operator/DISK4TB/Downloads
```

PASS: negative control: all 6 shipped entries ACCEPT with
QBITTORRENT_DATA_DIR=/home/milosvasic/Share/Misc

Unchanged by this session's work — reused, not re-implemented,
confirming the fence still accepts the live shipped configuration after
Case 6e landed.

Full regression sweep (zero regressions)

```
tests/unit/test_ownership_precondition.sh          32 passed, 0
failed, 0 skipped
tests/unit/test_ownership_repair.sh                153 passed, 0
failed, 0 skipped
tests/unit/test_ownership_probe_location.sh        10 passed, 0
failed, 0 skipped
tests/unit/test_ownership_gid_agreement.sh         5 passed, 0
failed, 0 skipped
tests/unit/test_ownership_rootless_detection.sh    19 passed, 0
failed
tests/unit/test_ownership_precondition_docker_premise.sh 4
passed, 0 failed, 0 skipped
tests/unit/test_ownership_repair_scope_tsv.sh      2 passed, 0
failed, 1 skipped (pre-existing, unrelated: HEAD moved past a
captured pre-fix baseline)
tests/unit/test_ownership_repair_setgid_strip.sh   1 passed, 0
failed, 1 skipped (pre-existing, unrelated: same class)
scripts/pre_build/check_cm_ownership_invariants.sh PASS: CM-
OWNERSHIP-INVARIANTS
```

bash -n tests/unit/test_ownership_precondition.sh — OK. shellcheck
is not installed on this host (probed, absent); the syntax check and the
full hermetic test run are the available evidence.

Files touched

- tests/unit/test_ownership_precondition.sh — added Case 6e
(paired §1.1 mutation), 109 lines. Nothing else in this file was
modified.
- scripts/ownership_precondition.sh — **unmodified**. The fence
(BOB-187's source-level fix) was already landed in commit af48019
prior to this session; confirmed byte-identical before and after
(sha256
f7e48c2394d5b2d7790497b8a47ca909bc2272774c082a4b80d3abb2ce6a72d
7).
- scripts/lib/ownership.sh — **unmodified**. ownership_path_fence()
already exists and is called by BOTH ownership_repair.sh and

ownership_precondition.sh (§11.4.251 — one predicate, never a second dialect); no change was needed or made.

Conclusion

BOB-187's source-level defect (the precondition creating probe files at an unfenced, .env-driven path) was already fixed prior to this session. The acceptance criterion this session closes is the previously-missing permanent paired §1.1 mutation regression test, landed as Case 6e. The suite now mechanically proves, on every run, that the fence call is genuinely wired into `main()` and load-bearing — not merely that a scope shaped like the escape happens to be refused for some other reason. Full negative control (6d) and RED-reproduction (6c) remain in force, unmodified.